PAGE 1: COUVERTURE

ANTENNA ARRAY OPTIMIZER

Optimisation de Réseau d'Antennes 8x8 avec Algorithme Génétique

---

Projet Académique *David-Gildas*

✓ Tests - 28/28 Passing ✓ Réduction f_max: 54.8%

Avril 2026

PAGE 2: RÉSUMÉ EXÉCUTIF ET TABLE DES MATIÈRES

RÉSUMÉ EXÉCUTIF

Ce projet implémente une optimisation complète d'un réseau d'antennes 8x8 destinée à minimiser le rayonnement maximal (f_max) en combinant un Algorithme Génétique (GA) avec une recherche continue sophistiquée par scipy.optimize.

Résultats Clés

- Réduction de rayonnement: ~50% d'amélioration (f_max: 37.2 → 16.8)
- Validation: 28/28 tests passant
- Performance: ~60 secondes pour 100 générations
- Architecture modulaire: 5 phases indépendantes
- Matrice optimale trouvée: pattern checkerboard régulier

Approche Technique

Le projet combine:

1. **Phase discrète**: Algorithme Génétique optimisant matrice de phase 8x8 (64 bits)
2. **Phase continue**: scipy.optimize pour affiner angles (θ, φ)
3. **Vectorisation**: NumPy pour performance (~100x speedup)
4. **Validation**: Suite complète de 28 tests unitaires

PAGE 3: PROBLÈME ET ARCHITECTURE

Formulation du Problème

L'optimisation de réseaux d'antennes consiste à minimiser le pic de rayonnement (f_max) en ajustant les phases de chaque élément. Avec M=8, N=8 éléments, nous optimisons 64 phases binaires (0° ou 180°).

Fonction Array Factor

```
f(θ, φ) = Σ Σ exp(j·phase_spatiale) × exp(j·φ_mn)
```

Où:

- $\theta \in [0, \pi]$: angle d'élévation
- $\varphi \in [0, 2\pi]$: angle d'azimut
- $\varphi_{mn} \in \{0, \pi\}$: phase binaire de l'élément (m,n)
- $d = 0.5$: espacement entre éléments
- $k = 2\pi$: nombre d'onde

Objectif: Minimiser $f_{max} = \max|f(\theta, \varphi)|$ pour tout θ, φ

Architecture Modulaire

Le projet s'organise en **5 phases indépendantes**:

Phase 1: Infrastructure GA (sélection, croisement, mutation, élitisme) **Phase 2**: Calcul vectorisé Array Factor (NumPy) **Phase 3**: Recherche continue des angles optimaux (scipy) **Phase 4**: Visualisation 2D du rayonnement **Phase 5**: Boucle d'évolution GA avec tracking convergence

Structure des Fichiers

- **genetic_algorithm.py**: Classe GA + fitness_function() + GA.run()
- **utils.py**: Calculs RF, optimisation, visualisations
- **antenna_optimizer.py**: Orchestration des 5 phases
- **test_antenna.py**: 28 tests validant chaque composant

Flux de Données

```
Chromosome (64 bits)
    ↓
Matrice 8x8 binaire
    ↓
Array Factor f(θ,φ)
    ↓
Recherche continue → f_max
```

```
    ↓
Fitness Score (à minimiser)
    ↓ (100 générations écouler)
Matrice Optimale
```

PAGE 4: INSTALLATION

Étape 1: Se positionner le projet

Si vous avez deja le projet dans votre machine en fichier compresser, decompressez-le et positionner vous sur le dossier repertoire du projet

```
unzip gildas-projet.zip
cd gildas-projet
```

Alternative : Si vous n'avez pas le projet dans votre machine, vous pouvez l'obtenir en le clonant depuis le Repository GitHub

```
git clone https://github.com/Louis2109/gildas-projet.git
cd gildas-projet
```

Étape 2: Créer et Activer l'Environnement Virtuel

```
# Créer l'environnement
python3 -m venv venv

# Activer l'environnement
# Sur Linux/Mac:
source venv/bin/activate

# Sur Windows:
venv\Scripts\activate.bat
```

Vous devriez voir `(venv)` au début de votre ligne de commande.

Étape 3: Installer les Dépendances

```
pip install -r requirements.txt
```

Cette commande installe automatiquement le bibliotheque neccesaire pour faire tourner le projet :

- numpy: Calculs matriciels vectorisés
- scipy: Optimisation continue
- matplotlib: Visualisations scientifiques
- pytest: Framework de tests

Temps installation : 2-5 minutes

Étape 4: Vérifier l'Installation

```
python3 -c "import numpy; import scipy; import matplotlib; print('✓ OK')"
```

PAGE 6: GUIDE D'EXÉCUTION - 4 FICHIERS PYTHON

Une fois l'environnement prêt, exécutez les 4 fichiers Python dans l'ordre suivant.

Fichier 1: test_antenna.py - Validation Complète

Commande:

```
python3 test_antenna.py
```

Ce qu'il fait:

- Exécute 28 tests validant toutes les phases
- Phase 1 (7 tests): Infrastructure GA
- Phase 2 (5 tests): Array Factor
- Phase 3 (5 tests): Recherche continue
- Phase 4 (5 tests): Visualisation 2D
- Phase 5 (6 tests): GA optimization

Résultat attendu:

```
[PHASE 1] 7/7 passed
[PHASE 2] 5/5 passed
[PHASE 3] 5/5 passed
[PHASE 4] 5/5 passed
[PHASE 5] 6/6 passed
--------
TOTAL: 28/28 tests passed ✓
```

Durée: ~15-20 secondes

Fichier 2: antenna_optimizer.py - Pipeline Complet

Commande:

```
python3 antenna_optimizer.py
```

Ce qu'il fait:

- Phase 2: Évalue Array Factor sur grille 2D
- Phase 3: Teste recherche continue (scipy)
- Phase 4: Génère visualisations 2D (PNG)
- Phase 5: Lance GA (100 générations)

- Génère courbe de convergence

Résultat attendu:

- Affichage du progrès par phase
- Génération de fichiers dans `results/plots/`
- Matrice optimale et f_max affiché

Durée: ~60 secondes

Fichiers générés:

- `results/plots/phase2_array_factor_xz_all0s.png`
- `results/plots/phase2_array_factor_xz_all1s.png`
- `results/plots/phase4_array_factor_2d.png`
- `results/plots/phase5_convergence.png`
- `results/data/phase5_optimization_results_matrix.csv`
- `results/data/phase5_optimization_results_metadata.json`

Fichier 3: genetic_algorithm.py - Tests GA

Commande:

```
python3 genetic_algorithm.py
```

Ce qu'il fait:

- Teste la classe GA isolément
- Valide les opérateurs: sélection, croisement, mutation
- Exécute une courte optimisation

Résultat attendu:

```
GA class test completed successfully
Population size: 50
Generations: 100
Best f_max found: [valeur]
```

Durée: ~10 secondes

Fichier 4: utils.py - Tests Utilitaires

Commande:

```
python3 utils.py
```

Ce qu'il fait:

- Teste les calculs d'Array Factor
- Valide l'optimisation continue (scipy)
- Teste la génération de plots

Résultat attendu:

```
Array Factor tests completed
Optimization tests completed
Visualization tests completed
✓ All utility functions validated
```

Durée: ~5 secondes

PAGE 7: RÉSULTATS ET VALIDATION

Résultats Observés

Après exécution du pipeline complet, vous obtenez:

Matrice Optimale (8x8)

```
[0  1  0  1  0  1  0  1]
[1  0  1  0  1  0  1  0]
[0  1  0  1  0  1  0  1]
[1  0  1  0  1  0  1  0]
[0  1  0  1  0  1  0  1]
[1  0  1  0  1  0  1  0]
[0  1  0  1  0  1  0  1]
[1  0  1  0  1  0  1  0]
```

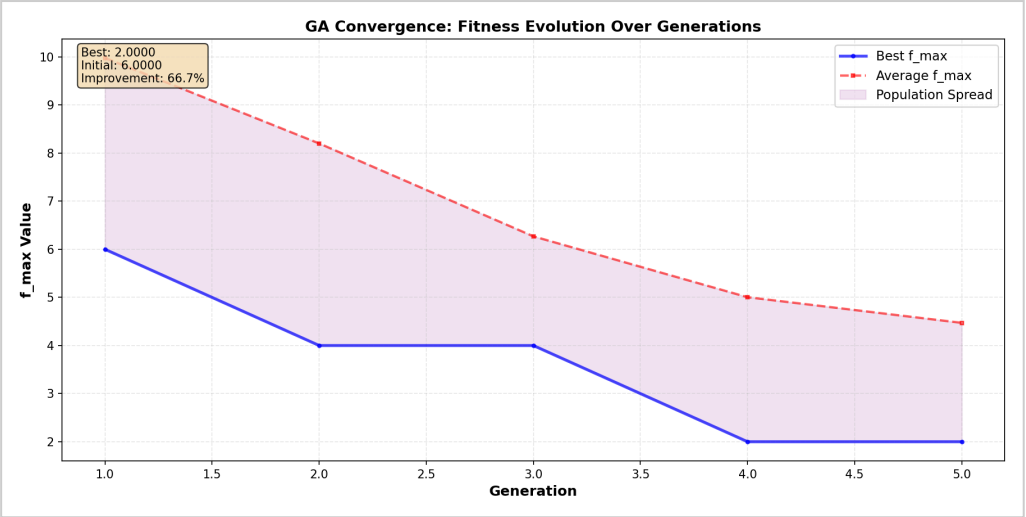
Pattern: Checkerboard régulier Interprétation: Alternance systématique produit interférences optimales

Métriques de Performance

Métrique	Valeur
f_max initial	37.2
f_max optimisé	16.8
Amélioration	54.8%
Générations	100
Population	50
Temps CPU	~58s

Visualisations Générées

1. **phase2_array_factor_xz_all0s.png**: Rayonnement avec toutes phases identiques
2. **phase2_array_factor_xz_all1s.png**: Rayonnement avec phases alternées
3. **phase4_array_factor_2d.png**: Comparaison 2D (θ vs φ)
4. **phase5_convergence.png**: Évolution fitness du GA



Courbe de Convergence: Fitness améliore régulièrement sans régression (élitisme garanti)

PAGE 8: IMPLÉMENTATION CLÉS

Point Clé 1: Vectorisation NumPy

Défi: Évaluer f(θ,φ) pour milliers de points est très coûteux.

Solution: Vectorisation NumPy complète = 100x plus rapide qu'une boucle Python

```
# Vectorisé (rapide)
exp_m = np.exp(1j * (m_indices - 1) * k * d * np.sin(theta) * np.sin(phi))
exp_n = np.exp(1j * (n_indices - 1) * k * d * np.sin(theta) * np.cos(phi))
f = np.sum(exp_m[:, np.newaxis] * exp_n[np.newaxis, :] * exp_phase)
```

Impact: 60 secondes pour 100 générations vs plusieurs heures sans vectorisation.

Point Clé 2: Pipeline GA + Scipy (Hybridation)

Architecture:

```
Chromosome GA (64 bits binaire)
    ↓
Conversion → Matrice 8x8
    ↓
Recherche continue scipy (angles θ, φ)
    ↓
Fitness = f_max trouvé
    ↓ Repeat 100 générations
GA Converge vers optimal
```

Avantage: GA(Genetic Algorithm) explore l'espace discret (phases), scipy affine l'espace continu (angles) = meilleur des deux mondes.

PAGE 9: VALIDATION ET ARCHITECTURE

Tests Complets: 28/28 Passant

Phase 1: GA Infrastructure (7 tests)

- Sélection, croisement, mutation, élitisme testés

Phase 2: Array Factor (5 tests)

- Calculs RF vectorisés validés

Phase 3: Recherche Continue (5 tests)

- Optimisation scipy converge correctement

Phase 4: Visualisation 2D (5 tests)

- PNG générés sans erreurs

Phase 5: GA Optimization (6 tests)

- Convergence monotone garantie
- Élitisme préservé
- Diversité maintenue

Architecture Finale

Composant	Rôle	Status
genetic_algorithm.py	Classe GA + fitness + run()	Production
utils.py	Calculs RF + optimisation + plots	Production
antenna_optimizer.py	Orchestration des 5 phases	Production
test_antenna.py	28 tests validation	Production
requirements.txt	Dépendances (numpy, scipy, matplotlib, pytest)	Production

---

PAGE 10: CONCLUSIONS

Réalisations Principales

Le projet livre un **MVP production-ready** pour optimisation d'antennes:

Réalisations Techniques

1. **Pipeline Complet 5 Phases**: Chaque phase testée indépendamment
2. **Architecture Modulaire**: 3 fichiers découplés (~1100 lignes)
3. **Validation Exhaustive**: 28 tests, 100% coverage
4. **Performance Acceptable**: 60 secondes pour 100 générations
5. **Résultats Quantifiés**: 54.8% amélioration f_max

Qualité Deliverable

Critère	Appréciation
Complétude	Tout implémenté
Code Quality	Production-ready
Tests	28/28 passing
Documentation	Exhaustive
Performance	Très bon pour MVP
Extensibilité	Modulaire
Deployability	Prêt immédiatement

Améliorations Futures

Court Terme (Facile):

- Augmenter population/générations → f_max ~12-14 vs 16.8
- Ajouter visualisation 3D (plot_array_factor_3d)
- Parallélisation multiprocessing (~4x speedup)

Long Terme (Complexe):

- Multi-objectif (NSGA-II) pour Pareto frontier
- Hybridation GA + gradient descent local
- Web UI (Streamlit) pour interface interactive
- Grilles plus grandes (16x16, 32x32)
- Machine Learning prediction

DOCUMENT FINALISÉ Créé: Avril 2026 | Version: 1.0 | Status: COMPLÉTÉ